

Práctica 2

Parte 1

La gestión del tiempo en los sistemas en tiempo real es fundamental para garantizar un funcionamiento eficiente, predecible y fiable de las aplicaciones. En este contexto, el tiempo no es simplemente una medida de duración, sino un recurso crítico que debe ser gestionado con precisión y eficacia.

Los STR se enfrentan a la tarea de procesar eventos y realizar acciones dentro de plazos estrictos, donde los retrasos o las interrupciones pueden tener consecuencias graves, como el fallo en el cumplimiento de una tarea crítica o la pérdida de datos importantes. Por lo tanto, la gestión del tiempo en estos sistemas implica la planificación, programación y coordinación de las actividades para asegurar que se completen dentro de los límites temporales especificados.

Para lograr esto, cualquier hardware de computación está provisto de temporizadores. Estos componentes son dispositivos que generan señales de tiempo precisas y pueden ser configurados para realizar diversas funciones, como la generación de interrupciones periódicas o la medición de intervalos de tiempo.

En los sistemas basados en ESP32 y ESP8266, el módulo de Timers proporcionado por ESP-IDF ofrece todas las funcionalidades y opciones de configuración que se necesitan para implementar aplicaciones en tiempo real. Veamos cómo se trabaja con él de forma práctica.

Sigue los siguientes pasos:

1. Crea el proyecto STR2024_P2. Crea la configuración “parte1_1”, tal y como se hizo en la última parte de la práctica anterior. En su fichero main.c copia el siguiente código:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_timer.h"
#include "esp_log.h"

#define MATRIX_SIZE 10
static const char* TAG = "P2_1_1";

void app_main(void)
{
    int matrix1[MATRIX_SIZE][MATRIX_SIZE];
    int matrix2[MATRIX_SIZE][MATRIX_SIZE];
    int result[MATRIX_SIZE][MATRIX_SIZE];

    // Inicializa las matrices con valores aleatorios
    for (int i = 0; i < MATRIX_SIZE; i++)
```

```

{
    for (int j = 0; j < MATRIX_SIZE; j++)
    {
        matrix1[i][j] = rand() % 10; // Valores aleatorios entre 0 y 9
        matrix2[i][j] = rand() % 10;
    }
}

// Tiempo inicial
uint64_t start_time = esp_timer_get_time();

// ... Multiplicación de las matrices ...
for (int i = 0; i < MATRIX_SIZE; i++)
{
    for (int j = 0; j < MATRIX_SIZE; j++)
    {
        result[i][j] = 0;
        for (int k = 0; k < MATRIX_SIZE; k++)
        {
            result[i][j] += matrix1[i][k] * matrix2[k][j];
        }
    }
}

// Tiempo final
uint64_t end_time = esp_timer_get_time();

// Tiempo transcurrido
double elapsed_time = end_time - start_time;

ESP_LOGI(TAG, "Matrix multiplication completed in %.2f <?>\n", elapsed_time);
}

```

El módulo de temporizadores se utiliza esencialmente para dos cosas: medir tiempos y configurar interrupciones temporizadas. Este código es un ejemplo representativo de lo primero ...

Recuerda hacer los cambios oportunos en Platformio.ini y los dos ficheros CMakeLists.txt para. En total son 3 cosas:

- En Platformio.ini: asignar la variable MODE de acuerdo a las configuraciones de esta práctica ("parte1_1", "parte1_2", etc.).
- En CMakeLists.txt más externo: hacer visible la variable MODE en el sistema de compilación de ESP-IDF.
- En CMakeLists.txt de src: en función del valor de MODE, asignar una carpeta de fuentes exclusiva a cada configuración del proyecto ("src/parte1_1", "src/parte1_2", etc.)

Apartado I: Cronómetros.

- 1.- Consulta la documentación de esp-idf y sustituye "<?>" por las unidades correctas en las que se está midiendo el tiempo de ejecución.
- 2.- Crea un menú en menuconfig para establecer el tamaño de la matriz.
- 3.- Lanza la prueba con diferentes tamaños y documenta el retardo de procesamiento medido.
- 4.- ¿Qué ocurre si le pones tamaños de matrices muy grandes?

2. Crea la configuración “parte1_2” del proyecto. En app_main, utiliza el siguiente código:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_timer.h"
#include "esp_log.h"

static const char* TAG = "P2_1_2";

// Callback function for the timer that occurs every second
void timer_callback_1(void* arg) {
    ESP_DRAM_LOGI(TAG, "Timer 1 Tick\n");
}

// Callback function for the timer that occurs every 5 seconds
void timer_callback_2(void* arg) {
    ESP_DRAM_LOGI(TAG, "Timer 2 Tick\n");
}

void app_main(void) {
    // Timer 1: estructuras manejador y configurador
    esp_timer_handle_t timer1;
    esp_timer_create_args_t timer_args1 =
    {
        .callback = &timer_callback_1, // callback de interrupción
        .name = "timer_1"
    };
    // creación del timer 1, se pasa manejador y configurador. Al terminar este método,
    // el manejador ya no es NULL
    esp_timer_create(&timer_args1, &timer1);
    // Uso del timer 1: interrupción periódica cada segundo (los timers están
    // implementados en microsegundos)
    esp_timer_start_periodic(timer1, 1000000); // 1 second in microseconds

    // Timer 2 configurado de forma análoga
    esp_timer_handle_t timer2;
    esp_timer_create_args_t timer_args2 =
    {
        .callback = &timer_callback_2,
        .name = "timer_2"
    };
    esp_timer_create(&timer_args2, &timer2);
    esp_timer_start_periodic(timer2, 5000000); // 5 segundos

    // Un bucle, para que la ejecución no se detenga
    while (1)
    {
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }

    // Lo que habría que hacer al finalizar con los timers de forma controlada.
```

... y este es un ejemplo representativo de lo segundo: usar Timers para controlar la temporización de la ejecución de nuestro código.

```
// No es necesario aquí porque la ejecución no llegará nunca a estas líneas
//esp_timer_delete(timer1);
//esp_timer_delete(timer2);
}
```

Apartado II: Alarmas.

1.- Modifica el código de los callbacks de interrupción para que impriman por pantalla el instante, desde el inicio de la ejecución, en que se produce el tick de cada uno de los timers. Deberemos encontrar algo como esto:

```
...
Timer1 Tick ( 1004513 us.)
Timer1 Tick (2009001 us.)
Timer1 Tick (3010234 us.)
Timer1 Tick (4012820 us.)
Timer 2 Tick(5050135 us.)
Timer1 Tick (5050210 us.)
...
```

2.- Observa la declaración de los callbacks. Tienen un argumento **void* arg**. Busca información por Internet sobre esto y modifica el código para hacer accesible dentro del callback una variable. En `app_main`, declara un contador para cada temporizador (**timer1_ticks** y **timer2_ticks**). Cada vez que suceda una expiración del contador, incrementa esta variable e imprime su valor por pantalla junto a la información que ya se ofrecía.

3.- Observa la última parte del código, donde hemos introducido un bucle improductivo para mantener la aplicación activa. En este caso se utiliza la función `vTaskDelay`. Busca información sobre esta función y responde:

- a) ¿Es una buena solución para temporizar código de forma precisa? ¿por qué?
- b) ¿Qué contiene la macro `portTICK_PERIOD_MS`?
- c) ¿Qué es el **tick del sistema** de FREERTOS?

4.- Sustituye el bucle final del código por `while(1);` (no te olvides del punto y coma). Esto es una espera activa, en la que la tarea no se bloquea nunca y permanece en ejecución para siempre.

- a) Compila, carga y lanza el código en la placa. ¿Qué ha ocurrido?.
- b) Haz un reset con el botón y cronometra externamente cuánto tiempo tarda en producirse el error.
- c) Accede a `menuconfig`. Navega a la opción `Component Config -> ESP System Settings -> Task Watchdog Timeout Period`, y modifica esta variable para poner 10 segundos. Compila, carga y lanza el código. Cronometra cuánto tiempo tarda en ocurrir el error.
- d) Busca información en Internet para responder: ¿Qué es el **WatchDog Timer** de un sistema?